

Zadanie indywidualne: Builder + Adapter w C#

Temat zadania

System generowania decyzji administracyjnej na podstawie danych z rejestru obywateli

Czas wykonania

Okolo **30–45 minut**

Poziom

Średni

Cel zadania

Celem zadania jest stworzenie aplikacji konsolowej w C#, która łączy dwa wzorce projektowe:

Builder

do stopniowego tworzenia złożonego obiektu decyzji administracyjnej.

Adapter

do podłączenia starego systemu rejestru obywateli, którego interfejs nie pasuje do nowego kodu.

W zadaniu należy pokazać, że wzorzec kreacyjny odpowiada za wygodne tworzenie obiektu, a wzorzec strukturalny za dopasowanie niezgodnego interfejsu.

Opis sytuacji

Nowy system administracyjny ma generować decyzje dla obywateli.

Decyzja administracyjna składa się z kilku części:

- numeru decyzji,
- danych obywatela,
- typu sprawy,
- podstawy prawnej,
- uzasadnienia,
- rozstrzygnięcia,
- pouczenia,
- daty wydania.

Problem polega na tym, że dane obywatela pochodzą ze starego systemu rejestrowego.

Stary system działa, ale zwraca dane w niewygodnej formie, np. jako jeden tekst:

44051401359;Jan Kowalski;Lublin

Nowa aplikacja nie chce pracować na takim tekście. Nowa aplikacja oczekuje obiektu:

Citizen

Dlatego należy zastosować **Adapter**, który przetłumaczy dane ze starego systemu na nowy model.

Następnie należy użyć **Buildera**, aby zbudować kompletną decyzję administracyjną krok po kroku.

Wymagania funkcjonalne

Program powinien:

1. Pobrać dane obywatela ze starego systemu rejestrowego.
2. Użyć Adaptera, aby zamienić dane tekstowe na obiekt Citizen.
3. Użyć Buildera, aby utworzyć obiekt AdministrativeDecision.
4. Wypisać gotową decyzję w konsoli.
5. Utworzyć minimum jedną decyzję administracyjną.

Wymagane klasy

W projekcie powinny znaleźć się następujące elementy.

1. Model obywatela

Utwórz klasę:

Citizen

Powinna zawierać minimum:

Pesel

FullName

City

2. Stary system rejestrowy

Utwórz klasę:

LegacyCitizenRegistry

Powinna mieć metodę, np.:

```
public string GetCitizenData(string pesel)
```

Metoda powinna zwracać dane obywatela jako tekst w formacie:

PESEL;Imię i nazwisko;Miasto

Przykład:

44051401359;Jan Kowalski;Lublin

To jest stary, niewygodny interfejs.

3. Nowy interfejs oczekiwany przez aplikację

Utwórz interfejs:

ICitizenProvider

Powinien zawierać metodę:

Citizen GetCitizen(string pesel);

Nowa aplikacja powinna korzystać właśnie z tego interfejsu, a nie bezpośrednio ze starego systemu.

4. Adapter

Utwórz klasę:

CitizenRegistryAdapter

Klasa powinna:

- implementować ICitizenProvider,
- korzystać wewnętrznie z LegacyCitizenRegistry,
- pobierać tekst ze starego systemu,
- rozdzielać tekst,
- tworzyć i zwracać obiekt Citizen.

Adapter ma ukryć przed resztą aplikacji fakt, że stary system zwraca dane jako tekst rozdzielony średnikami.

5. Model decyzji administracyjnej

Utwórz klasę:

AdministrativeDecision

Powinna zawierać minimum:

DecisionNumber

Citizen

CaseType

LegalBasis

Justification

Result

Instruction

IssueDate

Dodaj metodę:

Print()

która wypisze decyzję w konsoli.

6. Builder decyzji

Utwórz klasę:

AdministrativeDecisionBuilder

Builder powinien umożliwiać tworzenie decyzji krok po kroku, np.:

```
var decision = new AdministrativeDecisionBuilder()
    .WithDecisionNumber("DEC/2026/001")
    .ForCitizen(citizen)
    .WithCaseType("Świadczenie emerytalne")
    .WithLegalBasis("Ustawa o systemie ubezpieczeń społecznych")
    .WithJustification("Obywatel spełnia wymagania formalne.")
    .WithResult("Przyznaje się świadczenie.")
    .WithInstruction("Od decyzji przysługuje odwołanie w terminie 14 dni.")
    .Build();
```

Metody Buildera powinny zwracać obiekt buildera, aby można było stosować zapis łańcuchowy.

Oczekiwany sposób użycia w Main

W metodzie Main kod powinien wyglądać mniej więcej tak:

```
LegacyCitizenRegistry legacyRegistry = new LegacyCitizenRegistry();
```

```
ICitizenProvider citizenProvider = new CitizenRegistryAdapter(legacyRegistry);
```

```
Citizen citizen = citizenProvider.GetCitizen("44051401359");
```

```
AdministrativeDecision decision = new AdministrativeDecisionBuilder()
    .WithDecisionNumber("DEC/2026/001")
    .ForCitizen(citizen)
    .WithCaseType("Świadczenie emerytalne")
    .WithLegalBasis("Ustawa o systemie ubezpieczeń społecznych")
    .WithJustification("Obywatel spełnia wymagania formalne.")
    .WithResult("Przyznaje się świadczenie.")
    .WithInstruction("Od decyzji przysługuje odwołanie w terminie 14 dni.")
    .Build();
```

```
decision.Print();
```

Nazwy metod mogą być inne, ale program powinien wyraźnie pokazywać użycie Buildera i Adaptera.

Przykładowy wynik działania programu

DECYZJA ADMINISTRACYJNA
Numer decyzji: DEC/2026/001
Data wydania: 2026-06-16

Obywatel:
PESEL: 44051401359
Imię i nazwisko: Jan Kowalski
Miasto: Lublin

Typ sprawy:
Świadczenie emerytalne

Podstawa prawna:
Ustawa o systemie ubezpieczeń społecznych

Uzasadnienie:
Obywatel spełnia wymagania formalne.

Rozstrzygnięcie:
Przyznaje się świadczenie.

Pouczenie:
Od decyzji przysługuje odwołanie w terminie 14 dni.

Ograniczenia

Nie twórz całej decyzji przez jeden ogromny konstruktor z wieloma parametrami.
Nie korzystaj bezpośrednio z klasy LegacyCitizenRegistry w logice budowania decyzji.
Nie parsuj tekstu ze starego systemu w metodzie Main.
Nie umieszczaj całej logiki w jednej klasie.
Adapter powinien odpowiadać za dopasowanie starego systemu do nowego interfejsu.
Builder powinien odpowiadać za budowę decyzji administracyjnej.

Co należy oddać?

Kursant powinien oddać:

1. Projekt konsolowy C#.
2. Klasę LegacyCitizenRegistry.
3. Interfejs ICitizenProvider.

4. Klasę CitizenRegistryAdapter.
5. Klasę AdministrativeDecision.
6. Klasę AdministrativeDecisionBuilder.
7. Kod w Main, który tworzy decyzję i wypisuje ją w konsoli.
8. Krótką odpowiedź na pytania kontrolne.

Pytania kontrolne

Po wykonaniu zadania odpowiedz krótko:

1. Która klasa pełni rolę Adaptera?
2. Jaki problem rozwiązuje Adapter w tym zadaniu?
3. Która klasa pełni rolę Buildera?
4. Dlaczego Builder jest lepszy niż konstruktor z wieloma parametrami?
5. Co trzeba zmienić, jeśli stary system zacznie zwracać dane w innym formacie, np. JSON?
6. Czy klasa AdministrativeDecision powinna znać LegacyCitizenRegistry? Dlaczego?
7. Który wzorec jest kreacyjny, a który strukturalny?

Wariant dodatkowy dla szybszych osób

Dodaj drugi typ decyzji, np.:

Decyzja odmowna

Dla tej decyzji zmień:

- typ sprawy,
- uzasadnienie,
- rozstrzygnięcie,
- pouczenie.

Możesz też dodać drugiego obywatela w LegacyCitizenRegistry.

Przykład:

12345678901;Anna Nowak;Warszawa

Następnie wygeneruj dwie decyzje:

Decyzja pozytywna

Decyzja odmowna